

SISTEM MONITORING DAN KONTROL AC

SPEKIFIKASI DAN PRINSIP KERJA

Arsitektur, Mekanisme, dan Cara Kerja Sistem

KODE DOKUMEN	JZ-SMKA-SPEC-001
VERSI	1.0 2025-2026
PENGEMBANG	Jejak Zaidan — Production Studio

Dokumen ini bersifat RAHASIA. Hanya untuk penggunaan internal.

1. Gambaran Umum Sistem

1.1 Deskripsi Sistem

Sistem Monitoring dan Kontrol AC adalah sistem otomasi industri berbasis IoT yang dikembangkan oleh Jejak Zaidan. Sistem ini mengelola pendingin udara secara otomatis berdasarkan jumlah penghuni ruangan, sekaligus memantau konsumsi daya listrik secara real-time melalui tiga channel pengukuran independen.

Sistem bekerja tanpa memerlukan operator manual. Setiap perubahan jumlah orang di dalam ruangan memicu perhitungan ulang setpoint suhu dan pengiriman sinyal infrared ke unit AC secara otomatis. Data kondisi ruangan dan konsumsi daya dicatat secara simultan ke kartu SD (offline) dan Firebase Realtime Database (online).

1.2 Topologi Sistem

Jalur	Protokol	Keterangan	Interval
Slave ke Master	ESP-NOW 2.4 GHz	Laporan sensor: suhu, kelembaban, jumlah orang, status RFID	Tiap 2 detik
Master ke Slave	ESP-NOW 2.4 GHz	Perintah: data IR, enroll kartu, hapus kartu, ping keepalive	Event + tiap 5 det
Master ke Firebase	HTTPS via WiFi	Data realtime (ditimpa) dan historis (ditambah)	15 det / 5 menit
Master ke SD Card	SPI	Log CSV lokal dengan timestamp RTC	Tiap 5 detik

2. Spesifikasi Hardware

2.1 Master ESP32 — Pinout Lengkap

GPIO	Komponen	Arah	Keterangan Teknis
GPIO 4	IR Receiver TSOP1838	INPUT	Sinyal IR dari remote AC untuk mode learning, aktif LOW, 38 kHz carrier
GPIO 5	SD Card SPI (CS)	OUTPUT	Chip Select. SPI berbagi bus dengan tidak ada modul lain — dedicated
GPIO 16 (RX2)	PZEM-004T TX	INPUT	Diterima melalui voltage divider 10K+33K: tegangan output ±3.84V (aman untuk 3.3V GPIO)
GPIO 17 (TX2)	PZEM-004T RX	OUTPUT	Langsung tanpa level shifter — PZEM toleran 3.3V pada pin RX
GPIO 18 (SCK)	SD Card SPI	OUTPUT	Clock SPI — dikontrol oleh ESP32 sebagai master SPI
GPIO 19 (MISO)	SD Card SPI	INPUT	Data SD Card ke ESP32
GPIO 21 (SDA)	LCD I2C + RTC I2C	I/O	Diparalel: LCD 0x27 dan RTC DS3231 0x68, pull-up 4.7K
GPIO 22 (SCL)	LCD I2C + RTC I2C	OUTPUT	Diparalel dengan SDA
GPIO 23 (MOSI)	SD Card SPI	OUTPUT	Data ESP32 ke SD Card
GPIO 32	Button UP	INPUT	Pull-up eksternal 10K ke 3.3V, aktif LOW, debounce software 50ms
GPIO 33	Button DOWN	INPUT	Pull-up eksternal 10K ke 3.3V, aktif LOW, debounce software 50ms
GPIO 34	Button OK	INPUT	Input-only pin, TIDAK mendukung internal pull-up — wajib pull-up eksternal 10K

Catatan: GPIO 34 tidak memiliki internal pull-up/pull-down pada ESP32. Penggunaan `pinMode(34, INPUT_PULLUP)` tidak berefek — resistor pull-up eksternal 10K ke 3.3V adalah wajib.

2.2 Slave ESP32 — Pinout Lengkap

GPIO	Komponen	Arah	Keterangan Teknis
GPIO 4	IR Transmitter	OUTPUT	LED IR 940nm via resistor 100Ω. Dikontrol library <code>IRremoteESP8266</code> pada 38 kHz carrier
GPIO 5	RC522 RFID (CS/SDA)	OUTPUT	Chip Select RFID — pin berlabel SDA pada modul RC522, aktif LOW

GPIO	Komponen	Arah	Keterangan Teknis
GPIO 13	Proximity Sensor	INPUT	Output digital sensor proximity, aktif LOW, supply 5V (toleran 3.3V output oleh sensor ini)
GPIO 15	DHT22 DATA	I/O	Single-wire protocol, pull-up 10K ke 3.3V wajib, minimal interval baca 2 detik
GPIO 18 (SCK)	RC522 RFID SCK	OUTPUT	Clock SPI RFID
GPIO 19 (MISO)	RC522 RFID MISO	INPUT	Data RFID ke ESP32
GPIO 22 (RST)	RC522 RFID RST	OUTPUT	Reset RFID reader, aktif LOW
GPIO 23 (MOSI)	RC522 RFID MOSI	OUTPUT	Data ESP32 ke RFID
GPIO 25	LED Hijau	OUTPUT	Melalui resistor 220Ω ke GND. Indikator akses diterima dan mode enroll
GPIO 26	LED Merah	OUTPUT	Melalui resistor 220Ω ke GND. Indikator akses ditolak dan error
GPIO 27	Buzzer Aktif 5V	OUTPUT	Buzzer tipe aktif — berbunyi saat pin HIGH. Supply dari pin 3.3V cukup untuk buzzer aktif

2.3 PZEM-004T v3.0 — Spesifikasi Pengukuran

Parameter	Rentang Ukur	Resolusi	Akurasi
Tegangan AC (V)	80 — 260 VAC	0.1 V	+/- 0.5%
Arus AC (A)	0 — 100 A (via CT clamp)	0.001 A	+/- 0.5%
Daya Aktif (W)	0 — 23 kW	0.1 W	+/- 0.5%
Energi (kWh)	0 — 9999.99 kWh	1 Wh	+/- 0.5%
Power Factor	0.00 — 1.00	0.01	+/- 1%
Frekuensi (Hz)	45 — 65 Hz	0.1 Hz	+/- 0.5%

Komunikasi: UART TTL 9600 baud, protokol Modbus RTU. Tiga unit PZEM berbagi satu jalur serial (Serial2) dengan alamat Modbus berbeda: CH1=0x01, CH2=0x02, CH3=0x03. Master membaca ketiga unit secara berurutan dengan jeda 30ms antar channel.

3. Arsitektur Software

3.1 Master — Dual Core FreeRTOS

Master menggunakan dua core ESP32 secara paralel. Pembagian tugas dirancang agar operasi I/O lambat (PZEM, jaringan) tidak pernah memblokir antarmuka pengguna (LCD, button):

Core	Fungsi	Resource yang Diakses
Core 1 — loop()	State machine utama, polling button 50ms debounce, running text LCD, broadcast ping pairing, handle flag dari callback ESP-NOW	LCD, GPIO button, IR Receiver, ESP-NOW send
Core 0 — taskBackground()	Baca PZEM tiap 5 detik, push data ke Firebase (realtime 15det, historis 5menit), proses antrian SD card, ping slave keepalive tiap 5 detik, monitor heap tiap 30 detik	Serial2 (PZEM), WiFi/HTTP, SD Card, ESP-NOW send

Sinkronisasi antar core menggunakan empat FreeRTOS primitive:

- pzemMutex: SemaphoreHandle_t — melindungi array pzem_v[], pzem_i[], pzem_p[], pzem_pf[], pzem_e[] dari akses bersamaan. Core 0 mengambil mutex saat menulis hasil baca, Core 1 mengambil mutex saat membaca untuk display LCD. Timeout 300ms.
- sdMutex: melindungi operasi file SD card agar tidak ada dua write bersamaan yang dapat merusak FAT.
- lcdMutex: mencegah Core 0 (melalui logToSDQueue debug) dan running text engine mengakses LCD bersamaan. Timeout 100ms.
- sdQueue: QueueHandle_t berkapasitas 20 entry bertipe String*. Core 1 memproduksi pesan log (xQueueSend non-blocking), Core 0 mengonsumsinya (xQueueReceive non-blocking). Pointer String dialokasikan dengan new, di-delete setelah diproses.

3.2 State Machine Master — 17 State

Seluruh alur operasi master dikendalikan oleh satu variabel AppState. Setiap state mendefinisikan tampilan LCD, respons button, dan kondisi transisi:

State	Deskripsi	Tombol Aktif	Transisi
STATE_SLAVE_PAIRING	Pairing otomatis. Ditampilkan saat boot	OK=lewat	Slave found→IR_LEARNING; Timeout/OK→IDLE

State	Deskripsi	Tombol Aktif	Transisi
	pertama atau slaveCount=0		
STATE_IR_LEARNING	Instruksi rekam IR, tampilkan nama tombol yang perlu direkam (ON/OFF/TEMP_UP/TEMP_DOWN)	OK=mulai rekam	OK→IR_WAITING
STATE_IR_WAITING	Polling decode IR selama 10 detik. Tampilkan "Tunggu sinyal..."	Tidak ada	Sinyal OK→IR_LEARNING(next); Selesai→IDLE; Timeout→IR_LEARNING(ulang)
STATE_IDLE	4 view bergantian 5 detik: POWER CH1, CH2, CH3, DETAIL SLAVE	UP=prev, DOWN=next, OK=menu	OK→MENU
STATE_MENU	7 item menu dengan scroll. Menu timeout 7 detik	UP/DOWN=scroll, OK=pilih	OK→sub-state; Timeout→IDLE
STATE_MENU_ADD_SLAVE	Broadcast pairing dari menu. Timeout 60 detik	OK=batal	Found→IR_LEARNING; Timeout/OK→MENU
STATE_MENU_EDIT_SELECT	Pilih slave target untuk edit IR	UP/DOWN=pilih, OK=konfirmasi	OK→IR_LEARNING; Timeout→IDLE
STATE_MENU_ENROLL_SELECT	Pilih slave target untuk enroll kartu	UP/DOWN=pilih, OK=konfirmasi	OK→ENROLL_WAIT jika slave online
STATE_MENU_ENROLL_WAIT	Menunggu kartu RFID. Timer hitung mundur di baris 3 LCD	OK=batal	Kartu+feedback→NOTIF→MENU; Timeout→MENU
STATE_MENU_DELETE_ONE_WAIT	Menunggu kartu yang akan dihapus aksesnya	OK=batal	Kartu tempel→hapus+MENU; Timeout→MENU
STATE_MENU_RESET_CONFIRM	Konfirmasi hapus semua konfigurasi	OK=ya, UP=batal	OK→factoryReset+restart; UP→MENU
STATE_NOTIF	Tampilan notifikasi sementara. Durasi dikonfigurasi per pemanggil	Tidak ada	Timer habis→state berikutnya yang ditentukan

4. Mekanisme Pairing Master-Slave

4.1 Latar Belakang Masalah

ESP-NOW mengharuskan pengirim dan penerima berada di channel WiFi yang sama. Master terhubung ke router WiFi di channel tertentu (biasanya 1-13, tergantung router). Slave yang baru dinyalakan tidak mengetahui channel ini. Solusi naive menggunakan `esp_wifi_get_channel()` di sisi slave tidak akurat karena nilai yang dikembalikan adalah channel scan terakhir, bukan channel di mana paket diterima.

Selain itu, di lingkungan yang terdapat ESP32 lain, slave dapat secara keliru terpasang dengan perangkat yang bukan master sistem ini. Oleh karena itu digunakan protokol handshake dengan magic bytes.

4.2 Format Paket Ping Master

Paket ping master (9 byte):

```
Byte [0] = 0xAC    (magic byte 1)
Byte [1] = 0x57    (magic byte 2)
Byte [2] = WiFi.channel() (channel aktif master, misal 11)
Byte [3..8] = MAC address master (6 byte, dari esp_wifi_get_mac)
```

Magic bytes 0xAC 0x57 dipilih sebagai singkatan "AC System".

Probabilitas paket lain memiliki 2 byte pertama yang sama sangat rendah.

4.3 Alur Pairing Langkah demi Langkah

Langkah	Pelaku	Aksi	Detail
1	Master	Broadcast ping tiap 2 detik	Kirim 9 byte ke FF:FF:FF:FF:FF:FF selama STATE_SLAVE_PAIRING atau STATE_MENU_ADD_SLAVE aktif
2	Slave	Scan channel 1-13	Setup: <code>esp_wifi_set_channel(ch)</code> , tambah broadcast peer di channel itu, kirim SlavePayload (20 byte), tunggu 2 detik, ulangi ke channel berikutnya
3	Master	Terima SlavePayload dari slave baru	<code>onDataRecv</code> : cek <code>pairingWaiting==true</code> dan <code>slaveCount < MAX_SLAVES</code> . Jika ya: balas ke MAC slave dengan paket ping 9 byte yang sama (berisi channel aktif master)
4	Slave	Terima paket 9 byte	Validasi: <code>data[0]==0xAC && data[1]==0x57</code> . Jika valid: ekstrak <code>masterCh=data[2]</code> , <code>masterMac=data[3..8]</code>

Langkah	Pelaku	Aksi	Detail
5	Slave	Set channel dan simpan	esp_wifi_set_channel(masterCh), esp_now_add_peer(masterMac, masterCh), simpan ke NVS (masterMac, channel, masterKnown=true)
6	Slave	Kirim laporan pertama	reportNow=true di-set dari callback, loop() mengirim SlavePayload ke masterMac di masterCh
7	Master	Terima laporan, selesaikan pairing	slaveCount++, simpan MAC ke NVS, pairingWaiting=false, tampilkan notifikasi, transisi ke STATE_IR_LEARNING

4.4 Reconnect Otomatis Setelah Restart

Saat slave di-restart dengan master yang sudah dikenal (masterKnown=true di NVS):

1. Slave load masterMac dan channel dari NVS saat boot.
2. esp_wifi_set_channel(savedCh) — langsung set ke channel master tersimpan.
3. Kirim SlavePayload ke masterMac selama 10 detik.
4. Jika dalam 10 detik tidak ada respons ping dari master, anggap channel berubah dan mulai scan ulang semua channel.
5. Saat master restart, background task mengirim CMD_PING ke semua slave tersimpan. Slave menerima ping dan membalas — koneksi terbentuk kembali.

5. Mekanisme Penentuan Setpoint Suhu

5.1 Formula Perhitungan

Setpoint suhu dihitung secara deterministik setiap kali personCount berubah. Formula menggunakan model linear sederhana yang dapat dikonfigurasi:

```
calculateSetTemp(N) :  
  
    T_TARGET = 24.0 // suhu target untuk N_REF orang  
    K_FACTOR = 1.0  // penurunan suhu per orang tambahan (derajat Celsius)  
    N_REF     = 1    // jumlah orang referensi  
    T_MIN     = 18.0 // batas bawah (kemampuan minimum AC)  
    T_MAX     = 26.0 // batas atas (tidak terlalu dingin untuk 0 orang)  
  
    raw = T_TARGET - K_FACTOR * (N - N_REF)  
    t   = round(raw)           // bulatkan ke integer terdekat  
    t   = max(T_MIN, t)        // batasi bawah  
    t   = min(T_MAX, t)        // batasi atas  
    return t  
  
Contoh: N=3 → raw=24-1*(3-1)=22.0 → round=22 → clamp=22°C
```

Pembulatan ke integer dilakukan karena remote AC umumnya hanya menerima setpoint dalam kelipatan 1°C. Penggunaan nilai desimal akan mengakibatkan error perhitungan jumlah penekanan tombol TEMP_UP/DOWN.

5.2 Tabel Setpoint per Jumlah Orang

Jumlah Orang (N)	Perhitungan Raw	Setpoint Final	Status AC	Jumlah Penekanan dari 24°C
0	—	—	AC OFF (sendACOff())	—
1	24 - 1*(1-1) = 24.0	24°C	AC ON (sendACOn() + delay 3 detik)	0x TEMP_DOW N
2	24 - 1*(2-1) = 23.0	23°C	AC ON	1x TEMP_DOW N
3	24 - 1*(3-1) = 22.0	22°C	AC ON	2x TEMP_DOW N

Jumlah Orang (N)	Perhitungan Raw	Setpoint Final	Status AC	Jumlah Penekanan dari 24°C
4	$24 - 1 \cdot (4-1) = 21.0$	21°C	AC ON	3x TEMP_DOWN
5	$24 - 1 \cdot (5-1) = 20.0$	20°C	AC ON	4x TEMP_DOWN
6	$24 - 1 \cdot (6-1) = 19.0$	19°C	AC ON	5x TEMP_DOWN
7+	$24 - 1 \cdot (7-1) = 18.0$	18°C (clamp)	AC ON	6x TEMP_DOWN (maks)

5.3 Mekanisme Pengiriman Sinyal IR Penyesuaian

Saat setpoint berubah dari currentSet ke targetSet, slave menghitung selisih dan mengirimkan sinyal TEMP_UP atau TEMP_DOWN secara berulang:

```
adjustACTemp(currentSet, targetSet):  
  
    diff = round(targetSet - currentSet)  
  
    if diff > 0:  
        for i in range(diff):  
            sendTempUp()           // kirim raw IR data untuk tombol TEMP UP  
            vTaskDelay(500ms)      // jeda antar pengiriman (non-blocking)  
  
    elif diff < 0:  
        for i in range(abs(diff)):  
            sendTempDown()         // kirim raw IR data untuk tombol TEMP DOWN  
            vTaskDelay(500ms)  
  
    // vTaskDelay digunakan (bukan delay()) agar tidak trigger watchdog  
    // Asumsi: setiap 1 penekanan TEMP_UP/DOWN = perubahan 1°C pada AC
```

5.4 Logika Transisi On/Off AC

- N dari 0 ke 1: sendACOn() → vTaskDelay(3000ms) → adjustACTemp(T_MAX, 24°C). Delay 3 detik diperlukan agar AC sempat menyala dan siap menerima perintah perubahan suhu.
- N dari 1 ke 0: sendACOff() → setTemp = T_TARGET. Setpoint direset ke nilai default agar saat orang masuk lagi perhitungan dimulai dari awal.

- N bertambah (1→2, 2→3, dst): adjustACTemp(setTemp_lama, setTemp_baru) → AC tidak dimatikan, hanya setpoint diturunkan.
- N berkurang (3→2, 2→1, dst): adjustACTemp(setTemp_lama, setTemp_baru) → Setpoint dinaikkan, AC tidak dimatikan.

6. Mekanisme Person Counter

6.1 Arsitektur Dual Sensor

Person counter menggunakan dua sensor yang bekerja secara komplementer:

- RFID RC522 sebagai sensor masuk: Setiap orang yang masuk ruangan menempelkan kartu RFID terdaftar. Sistem mencatat UID kartu ke array `insideCards[]` dan menambah `personCount`.
- Proximity Sensor sebagai sensor keluar: Dipasang di pintu keluar. Setiap kali sensor mendeteksi orang melewatinya (transisi HIGH ke LOW, debounce 300ms), sistem mengurangi `personCount` dan menghapus kartu pertama dari `insideCards[]` (FIFO).

6.2 Inside Tracking — Pencegahan Duplikasi

Array `insideCards[MAX_CARDS][4]` menyimpan UID kartu yang saat ini berada di dalam ruangan. Setiap tap kartu divalidasi terhadap array ini sebelum menambah counter:

Saat RFID terbaca:

```
if cardExists(uid):           // UID ada di database terdaftar?
    if cardInsideExists(uid):  // UID sudah tercatat di dalam?
        feedbackDuplicate()    // 2 kedip cepat — abaikan
        return
    else:
        personCount++
        addCardInside(uid)     // tambahkan ke insideCards[]
        feedbackEnter()        // 1 beep panjang
        adjustACTemp(...)
    else:
        feedbackError()        // kartu tidak dikenal — 3 beep
```

Saat Proximity trigger:

```
if personCount > 0:
    personCount--
    removeFirstCardInside()    // FIFO: hapus kartu yang masuk paling awal
    adjustACTemp(...)
```

6.3 Keterbatasan Person Counter

- Asumsi 1:1 — satu orang satu kartu. Jika satu orang memiliki lebih dari satu kartu dan menggunakan keduanya, counter akan salah.
- Sensor proximity hanya mendeteksi satu orang per trigger. Jika dua orang keluar bersamaan, hanya satu yang terhitung.

- FIFO untuk keluar tidak memperhitungkan kartu siapa yang keluar — hanya mengurangi counter dan menghapus kartu yang paling lama di dalam. Ini akurat jika orang keluar berurutan, tetapi mungkin tidak akurat jika terjadi edge case.

7. Format Komunikasi dan Protokol

7.1 SlavePayload — Laporan Slave ke Master

Field	Tipe Data	Ukuran	Nilai	Deskripsi
temp	float	4 byte	Contoh: 26.5	Suhu ruangan dari DHT22 dalam derajat Celsius
hum	float	4 byte	Contoh: 58.3	Kelembaban relatif dari DHT22 dalam persen
personCount	int	4 byte	0 hingga MAX_CARDS	Jumlah orang yang sedang di dalam ruangan
setTemp	float	4 byte	18.0 hingga 26.0	Setpoint suhu AC aktif (nilai integer dalam float)
cardCount	uint8_t	1 byte	0 hingga 10	Jumlah kartu RFID yang terdaftar di database slave
enrollStatus	uint8_t	1 byte	0/1/2/3	0=idle, 1=sukses, 2=duplikat, 3=penuh

7.2 MasterCommand — Perintah Master ke Slave

Field	Tipe Data	Ukuran	Nilai/Deskripsi
cmd	uint8_t	1 byte	0x01=CMD_SEND_IR, 0x02=CMD_ENROLL_START, 0x03=CMD_DELETE_CARDS, 0x04=CMD_PING, 0x05=CMD_DELETE_ONE_CARD
slaveIdx	uint8_t	1 byte	Index slave tujuan (0 sampai MAX_SLAVES-1)
irData[100]	uint32_t[100]	400 byte	Array raw timing sinyal IR dalam mikrodetik. Diisi hanya untuk CMD_SEND_IR
irLen	uint16_t	2 byte	Jumlah elemen valid dalam irData (biasanya 67-71 elemen)
irType	uint8_t	1 byte	0=IR_ON, 1=IR_OFF, 2=IR_TEMP_UP, 3=IR_TEMP_DOWN
cardUid[4]	uint8_t[4]	4 byte	UID kartu target untuk operasi hapus 1 kartu

7.3 Penanganan Callback ESP-NOW

Seluruh aksi yang memerlukan `delay()` atau operasi lambat DILARANG dipanggil langsung dari dalam `onDataRecv()` ESP-NOW callback. Callback berjalan di interrupt context — `delay()` di dalamnya akan menyebabkan Interrupt WDT Timeout (Guru Meditation Error).

Sistem menggunakan pola flag untuk menunda eksekusi ke `loop()`:

- `reportNow`: jika true, `loop()` akan memanggil `reportToMaster()` di iterasi berikutnya

- `irAllReady`: jika `true`, `loop()` akan memanggil `feedbackOK()` dan `sendACOn()`
- `enrollFeedback`: jika `true`, `loop()` akan memanggil `feedbackEnrollMode()`
- `deleteFeedback`: jika `true`, `loop()` akan memanggil `feedbackError()`

8. Penyimpanan Data

8.1 Firebase Realtime Database

Struktur Data Realtime (path /realtime, diperbarui tiap 15 detik)

```
/realtime/
  pzem1/ { v: 220.5, i: 1.23, p: 270.1, pf: 0.98, e: 0.125 }
  pzem2/ { v: 220.3, i: 0.00, p: 0.0, pf: 0.00, e: 0.001 }
  pzem3/ { v: 220.1, i: 0.00, p: 0.0, pf: 0.00, e: 0.001 }
  Slave 1/ { temp: 26.5, hum: 58.3, n: 2, tset: 23, cards: 3, online: true
}
```

Struktur Data Historis (path /history, ditambahkan tiap 5 menit)

```
/history/
  2025-03-15/
    09:30/ { pzem1/{v,i,p,e}, pzem2/{v,i,p,e}, pzem3/{v,i,p,e},
             Slave 1/{temp, hum, n, tset} }
    09:35/ { ... }
    09:40/ { ... }
  2025-03-16/
    ...
```

Data historis > 30 hari dihapus otomatis tiap jam oleh master.

8.2 SD Card — Log CSV

Struktur direktori: /YYYY/MM/log.csv

Format baris PZEM:

```
2025-03-15T09:30:00,PZEM1,220.5,1.23,270.1,0.98,0.125
// timestamp, channel, V, I, P, PF, E
```

Format baris slave:

```
2025-03-15T09:30:00,Slave 1,T:26.5,H:58.3,N:2,Tset:23
```

Logging melalui sdQueue (kapasitas 20 String*)

Jika queue penuh, data dibuang (non-blocking xQueueSend)

8.3 NVS — Penyimpanan Konfigurasi Permanen

Namespace	Key	Tipe	Konten
slaves (master)	count	int	Jumlah slave yang tersimpan
slaves (master)	s0 .. s4	bytes	Struct SlaveConfig per slave: MAC[6], label[16], irON[100], irOFF[100],

Namespace	Key	Tipe	Konten
			irTEMP_UP[100], irTEMP_DOWN[100], panjang masing-masing
sys (master)	firstrun	bool	true=belum dikonfigurasi, false=sudah
sys (slave)	masterKnown	bool	Apakah master sudah dikenal
sys (slave)	masterMac	bytes	MAC address master (6 byte)
sys (slave)	channel	uint8	Channel WiFi master yang tersimpan
cards (slave)	count	uint8	Jumlah kartu terdaftar
cards (slave)	c0 .. c9	bytes	UID per kartu (4 byte)
irdata (slave)	irON/OFF/UP/DN	bytes	Data raw IR timing tersimpan (uint32_t array)

9. Dashboard Web

9.1 Fitur dan Tampilan

Fitur	Deskripsi Teknis
Status Ruangan	Render dinamis card per slave dari /realtime/. Badge online/offline berdasarkan field online di Firebase
Tabel PZEM	Enam kolom: V, I, P, PF, E per channel. Auto-refresh setInterval 15 detik via fetch ke Firebase REST API
Grafik Historis	Chart.js line chart, 4 grafik: suhu, daya, orang, kelembaban. Mengambil data dari /history/YYYY-MM-DD per tanggal dalam rentang filter
Filter Preset	1 Hari, 1 Minggu, 1 Bulan — otomatis set dateTo=hari ini, dateFrom=N hari lalu
Date Range Kustom	Input type=date untuk from dan to, validasi from<=to sebelum fetch
Export XLSX	SheetJS: Sheet 1=data raw semua kolom, Sheet 2=ringkasan statistik (avg, min, max suhu, kelembaban, daya)
Export PDF	jsPDF + AutoTable: layout landscape A4, halaman 1=ringkasan, halaman 2+=tabel data (max 500 baris)
Export PNG	canvas.toDataURL("image/png") per grafik Chart.js. Export semua = 4 file PNG diunduh berurutan

10. Keamanan dan Keterbatasan

10.1 Keamanan Implementasi

Lapisan	Mekanisme	Deskripsi
Pairing	Magic bytes 0xAC 0x57	Mencegah slave terpasang dengan perangkat ESP-NOW lain secara keliru
Akses Fisik	RFID UID validation	Hanya UID yang ada dalam database NVS slave yang diterima
Anti-duplicate	insideCards[] tracking	Kartu yang sama tidak bisa menambah counter dua kali
Cloud	Firebase Database Rules	Write hanya dengan auth token (Database Secret), read publik

10.2 Keterbatasan Diketahui

Keterbatasan	Dampak	Mitigasi
Jangkauan ESP-NOW 10-20m dalam ruangan	Slave jauh tidak terjangkau	Site survey sebelum instalasi, gunakan repeater jika perlu
Maks 5 slave per master	Tidak bisa scale ke banyak ruangan dengan 1 master	Deploy multiple master unit dengan Firebase yang sama

Keterbatasan	Dampak	Mitigasi
RC522 tidak mendukung e-KTP (ISO 14443B)	e-KTP tidak bisa digunakan sebagai kartu akses	Gunakan kartu Mifare Classic atau ganti ke reader PN532
RTC butuh baterai CR2032	Tanpa baterai, waktu reset ke 2000 saat mati listrik	Pastikan baterai terpasang dan berfungsi sebelum deployment
Proximity sensor: 1 orang per trigger	Dua orang keluar bersamaan hanya terhitung 1	Gunakan dua sensor proximity (masuk + keluar) untuk akurasi tinggi